

Does the Idea-Merging Engine Work? A Plain-Language Validation Report

Testing how Freedi groups many written opinions into a few clear ideas

June 2026

Contents

What this system does (in one paragraph)	1
How we tested it	1
What we found	2
Bugs we found and fixed	4
Conclusions	4
What we have <i>not</i> yet proven (open questions)	5
How anyone can reproduce this	5

What this system does (in one paragraph)

Freedi collects free-text answers from many people responding to a question (for example, “*What should our country do?*”). Two people often say the same thing in different words. The job of the **synthesis engine** is to read all the answers and:

1. **Merge** answers that mean the same thing into a single combined statement (we call each combined statement a “**synth**”).
2. **Keep apart** answers that are genuinely different — even when they sound similar.
3. **Keep opposites apart** — “make peace” and “do *not* make peace” must never be merged.
4. **Leave a lone, unrelated answer on its own** instead of forcing it into a group.
5. **Group the synths into broader topics**.

This report asks one question: **does the engine actually do these five things correctly?**

How we tested it

We cannot judge correctness on real data, because with real data nobody knows the “right” answer in advance. So we built **five test datasets where we already knew the correct grouping** — we wrote the answers ourselves and labelled which ones belong together. Then we ran the engine and checked whether its output matched the grouping we had built in.

A few terms used throughout, in plain language:

- **Embedding** — each sentence is turned into a list of numbers that captures its meaning. Sentences with similar meaning get similar numbers.
- **Cosine similarity** — a score from 0 to 1 measuring how close two sentences’ meanings are (1 = identical meaning, 0 = unrelated). It is computed from the embeddings, with no AI judgement.
- **Two-stage judge** — the engine first uses the cheap cosine score. Pairs that are clearly very similar or clearly very different are decided immediately. Only the **uncertain middle** is sent to an AI language model, which reads the two sentences and rules “same idea” or “different idea.” This keeps cost low while still catching the hard cases.

Important honesty note (applies to every test below): all five datasets were *written by an AI*, not collected from real human debate. AI-written sentences are cleaner and more on-topic than real human input. So passing these tests is **necessary but not sufficient** — it shows the mechanism is sound, but real-world robustness still needs to be checked on real data.

The five datasets

Dataset	What it contains	What it tests
Reference	2 topics, 4 ideas, 40 answers	Basic recovery — the clean baseline
Loose wording	3 topics, 12 ideas, 60 answers, worded <i>very</i> differently	Recovery when paraphrases barely resemble each other
Tight wording	Same as above, but paraphrases worded similarly	Recovery under realistic wording
Opposites	2 subjects, each with a <i>pro</i> and an <i>anti</i> stance, 20 answers	Opposites must not be merged
Lone outlier	40 on-topic answers + 1 unrelated one	The unrelated answer must be left alone

The “loose” vs “tight” pair is the key experiment: **same ideas, different wording width**. It isolates the effect of *how differently people phrase the same idea* from the effect of the mechanism itself.

What we found

Headline results

Dataset	Result	Plain-language verdict
Reference	PASS	All 4 ideas recovered cleanly; 2 correct topics; no overlap.

Dataset	Result	Plain-language verdict
Loose wording	Partial fail	The hardest distinction (two close ideas) was made correctly, but half the ideas got split into fragments because the wording was <i>unrealistically</i> varied.
Tight wording	PASS	Perfect: all 12 ideas recovered, every answer placed, no overlap — and the AI judge wasn’t even needed, because cosine alone handled it.
Opposites	PASS (now perfect)	All 4 stances kept separate with zero mixing; after a fix (below), all 20 answers correctly placed .
Lone outlier	PASS	The unrelated answer was correctly left on its own .

The three things this proved

1. **On realistically-worded input the engine is exact.** In the tight-wording test it recovered every idea perfectly, placed every answer, and produced no overlaps — using only the cheap cosine step, with no AI calls at all.
2. **The AI judge does what the cosine score cannot.** Cosine similarity alone *merges opposites*, because “make peace” and “do not make peace” use almost identical words and so score as highly similar. Only the AI language model, reading the actual meaning, separates them. In the opposites test, a cosine-only method produced 2 (wrong, conflated) groups; the full two-stage judge produced the **4 correct, stance-pure** groups.
3. **A lone outlier is left alone.** Even though the proximity-based step initially pulled the unrelated answer toward the nearest group, the judge correctly excluded it.

The one real limitation we observed

The dominant cause of any failure was **how the test sentences were worded**, not the mechanism. When we deliberately wrote paraphrases of one idea using *maximally different* words (the “loose” dataset), their meaning-similarity dropped below the engine’s confidence threshold, every answer had to go through the AI judge, and some ideas fragmented. When we wrote the *same ideas* with normal wording overlap (the “tight” dataset), recovery jumped from half the ideas to **all** of them. Real human near-duplicates share more wording than our “loose” set, so they would behave like the “tight” case.

Bugs we found and fixed

Running these controlled tests exposed **three genuine defects** in the live system (not test-only problems). Each was fixed and locked in with an automated regression test so it cannot silently return.

1. **An answer could be counted in two groups at once.** A quirk in the proximity-clustering library let a borderline answer be claimed by two groups. Fixed so every answer lands in exactly one group.
2. **A confidence threshold was set too aggressively.** The cut-off for “too different to bother asking the AI” was set so high that genuinely-similar paraphrases were being thrown out *without ever being checked*. We lowered it to a true “clearly unrelated” floor, so the AI now arbitrates the uncertain middle instead of the engine discarding answers prematurely.
3. **A high-agreement group kept its disagreeing members.** When most of a group agreed, the engine was keeping the *entire* group — including the few members that had actually been judged not to belong. Fixed so a kept group contains **only** the members that genuinely agree.

A fourth refinement: rescuing good answers dropped by one stray AI verdict

The AI judge is not perfectly consistent — occasionally it wrongly rules one pair of genuinely-matching sentences “different.” The old rule required a member to match **every** other member of its group unanimously, so a single stray “different” verdict could wrongly **eject a correct answer** from its group.

We replaced the unanimity rule with a **majority rule** (technically, a *quorum*): an answer joins a group if it matches a clear majority of the group’s members, tolerating one stray disagreement. Crucially, this does **not** let opposites slip in — a genuinely opposite answer disagrees with the *whole* opposing group, so it can never reach a majority. The change adds **no extra AI cost** (it re-uses comparisons already made).

Effect on the opposites test: two correct answers that the old rule had dropped were recovered. The score went from **17 of 20** answers correctly placed to **all 20**, and we confirmed this result was **identical across three repeated runs** — with opposites still never merged.

Conclusions

1. **Idea-merging is correct and trustworthy on realistically-worded input** — exact recovery, full coverage, no overlap.
 2. **The AI judge is essential for the hard cases** cosine cannot handle: separating two close-but-distinct ideas, and separating opposite stances that look similar on the surface.
 3. **Lone outliers are correctly left unclustered.**
 4. **Wording realism is the single biggest factor** in success — and our hardest test deliberately used unrealistic wording, so it understates real-world performance.
 5. **Two settings depend on the dataset and must always be reported, never quietly tuned:** the clustering “neighbourhood size” (which controls whether you get fine ideas or broad topics) and the cap on how many AI calls are allowed.
-

What we have *not* yet proven (open questions)

- **Real human data.** Every test sentence was AI-written. The decisive next step is to repeat this on genuine human input.
- **Grouping closely-related topics.** The engine cleanly grouped clearly-distinct topics (e.g. fitness vs. finance), but the *diagnostic* method we used for topic-grouping chained together very closely-related sub-topics. The production topic-grouping path works differently and was not exercised here. **This is the main open item.**
- **Occasional recall loss from comparing against a single reference point.** The judge compares each candidate against one central member of a group. If that one comparison is noisy, a good answer can still be dropped. The majority rule above fixes the *single* stray verdict; cases where an answer is dropped by *several* noisy verdicts remain a target for future improvement.

How anyone can reproduce this

Every result can be re-checked at three levels of effort, from easiest to most complete:

1. **Re-score the saved results** — no special software or keys needed.
2. **Re-run the deterministic clustering** from the saved meaning-vectors — this reproduces the grouping exactly, without needing the AI services.
3. **Run the whole pipeline end-to-end** — requires a local database and access to the AI services.

The first two levels are fully deterministic and reproducible by anyone with the published data files. The AI-judge step is, by nature, not perfectly deterministic, which is why the opposites result above was repeated three times to confirm stability.

*This is a plain-language summary. The full technical report — with exact parameters, commit hashes, file paths, and per-run details — is in **FINDINGS.md** and the per-run **report.md** files. Part of the Freedi project, released under the GPL-3.0 license.*